Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине «ЖИЗНЕННЫЙ ЦИКЛ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ»
Вариант №1

Студент: Р.А. Адамонис

Преподаватель: Д.В. Куприянова

МИНСК 2026

ЦЕЛЬ РАБОТЫ

Изучить принципы работы с системой контроля версий Git, освоить основные операции по созданию, настройке и управлению локальными и удалёнными репозиториями, а также получить практические навыки организации совместной разработки программного проекта с использованием платформы GitHub.

ВЫПОЛНЕНИЕ РАБОТЫ

На начальном этапе выполнения лабораторной работы была выполнена установка системы контроля версий Git, необходимой для отслеживания изменений в проекте, ведения истории разработки и взаимодействия с удалёнными репозиториями. После установки была проведена проверка доступности Git и Java, а также соответствия их версий требованиям дальнейшей работы. Результаты проверки представлены на рисунке 1.

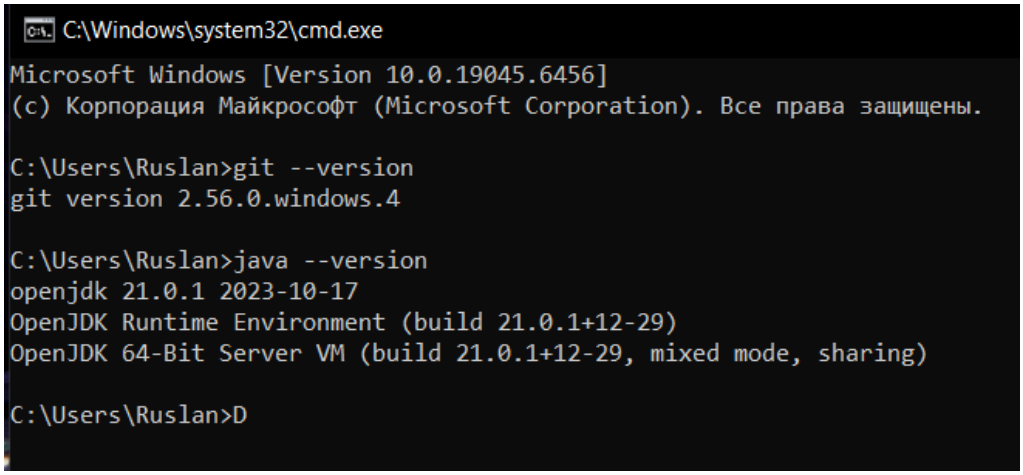


Рисунок 1 – Проверка установленных версий Java и Git

Далее была выполнена первоначальная настройка Git. Для идентификации автора изменений в истории репозитория заданы глобальные параметры имени пользователя и адреса электронной почты. После настройки дополнительно проверен список параметров, что позволило убедиться в корректности сохранения указанных данных (рисунок 2).

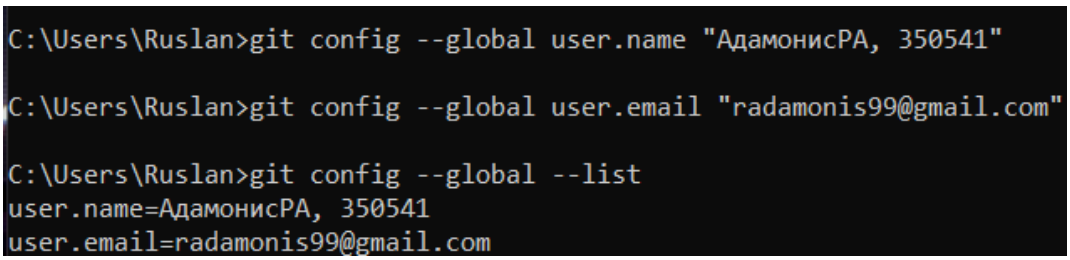


Рисунок 2 – Настройка глобальных параметров Git

Для взаимодействия с удалённым репозиторием GitHub была настроена SSH-аутентификация. Сначала проверена доступность SSH-клиента и его версия (рисунок 3). SSH (Secure Shell) – протокол, обеспечивающий защищённое соединение с удалёнными сервисами. В рамках лабораторной работы он используется для безопасной авторизации на GitHub.

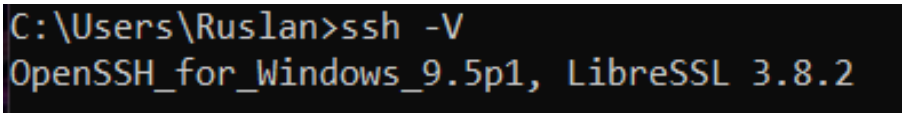


Рисунок 3 – Проверка наличия SSH-клиента и его версии

Следующим шагом стала генерация пары SSH-ключей с использованием алгоритма Ed25519. В процессе создания был указан путь сохранения файлов и задана парольная фраза для дополнительной защиты закрытого ключа. В результате были созданы открытый и закрытый ключи, необходимые для последующей аутентификации (рисунок 4).

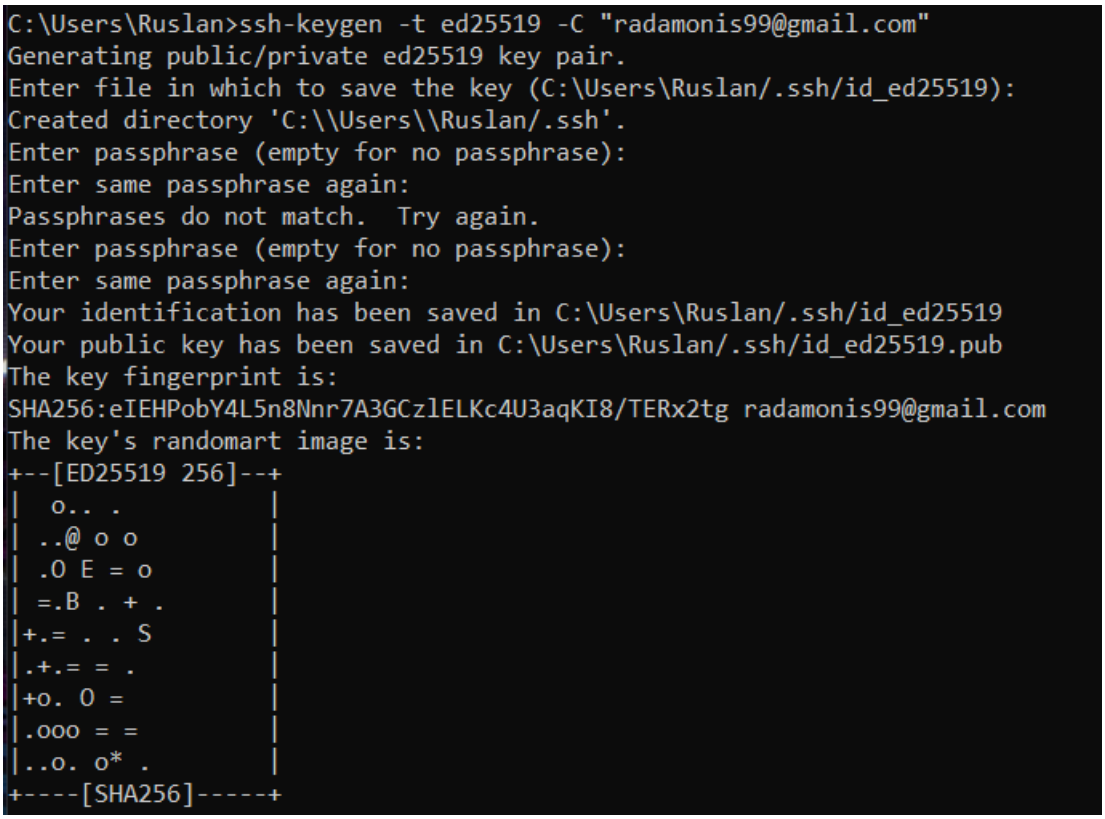


Рисунок 4 – Генерация пары SSH-ключей

После генерации была проверена папка .ssh, предназначенная для хранения ключей и конфигурационных файлов SSH. В её содержимом обнаружены созданные файлы открытого и закрытого ключей, что подтвердило успешное выполнение предыдущего этапа (рисунок 5).

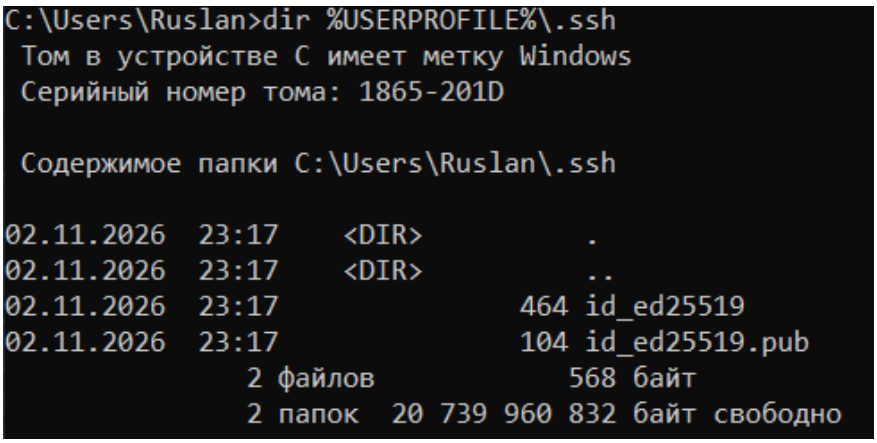


Рисунок 5 – Проверка содержимого каталога SSH

Далее была настроена служба SSH Agent, предназначенная для управления закрытыми ключами и хранения их в памяти во время пользовательского сеанса. Для удобства работы службе установлен автоматический запуск, после чего она была запущена (рисунок 6).

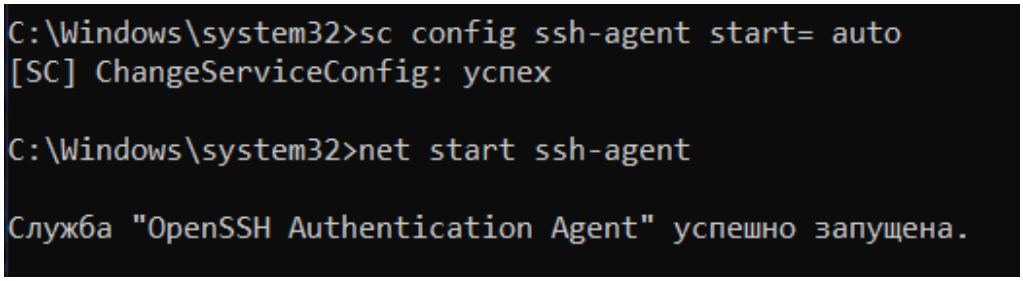


Рисунок 6 – Настройка и запуск службы SSH Agent

После запуска агента в него был добавлен созданный закрытый ключ. При добавлении указана ранее заданная парольная фраза. Успешное выполнение операции подтвердило, что ключ загружен и готов к использованию при установлении SSH-соединений (рисунок 7).

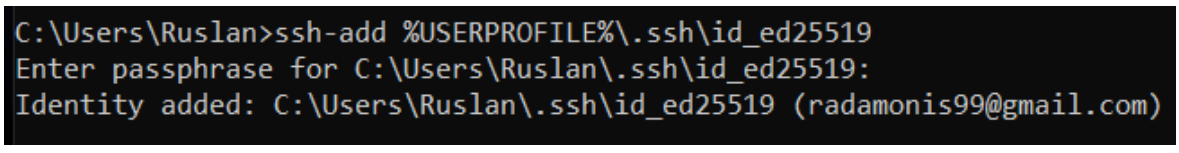


Рисунок 7 – Добавление SSH-ключа в SSH Agent

Для проверки наличия загруженного ключа был просмотрен список ключей SSH Agent. Отображение информации о ключе и его отпечатке подтвердило успешную загрузку и готовность агента к выполнению аутентификации (рисунок 8).



Рисунок 8 – Проверка загруженных SSH-ключей

Завершающим шагом на данном этапе стала проверка подключения к GitHub по SSH. В результате сервер успешно распознал зарегистрированный открытый ключ и подтвердил аутентификацию пользователя. Полученное сообщение свидетельствует о корректной настройке защищённого соединения и возможности дальнейшей работы с удалёнными репозиториями (рисунок 9).

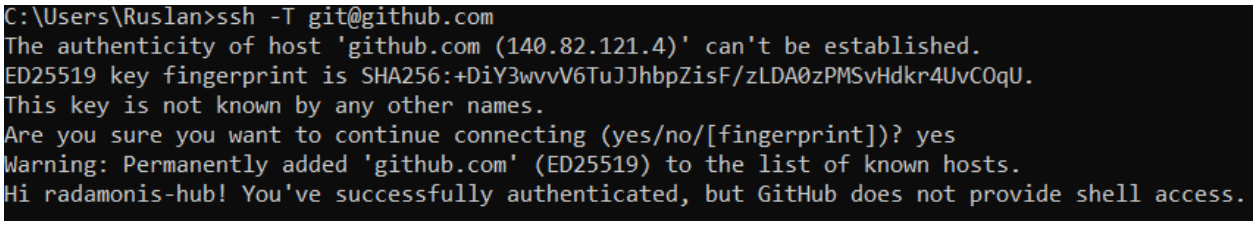


Рисунок 9 – Проверка SSH-аутентификации на GitHub

Для расширения возможностей взаимодействия с GitHub через командную строку была установлена утилита GitHub CLI. Она позволяет выполнять операции с учётной записью и удалёнными репозиториями непосредственно из терминала, что упрощает выполнение последующих этапов лабораторной работы. Установка инструмента представлена на рисунке 10.

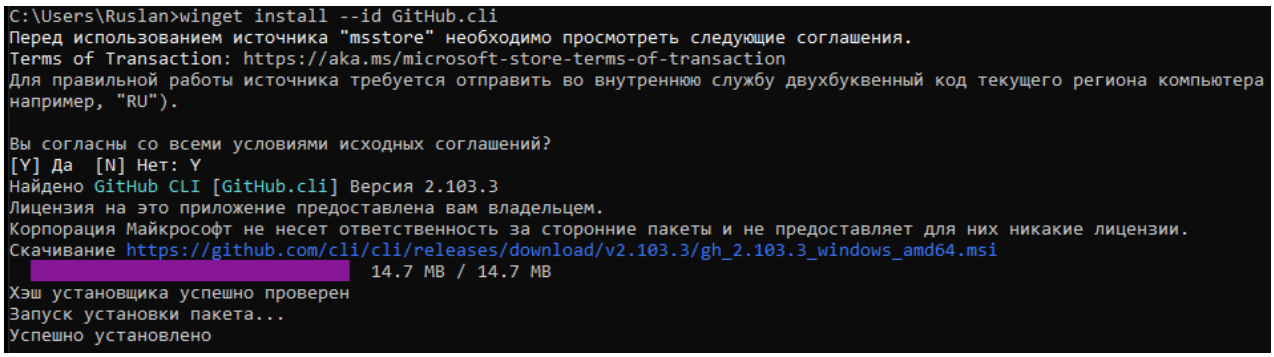


Рисунок 10 – Установка GitHub CLI

После установки GitHub CLI была проверена корректность его установки и доступность утилиты в командной строке. Проверка версии позволила убедиться, что инструмент установлен и готов к дальнейшему использованию (рисунок 11).

```
C:\Users\Ruslan>gh --version
gh version 2.103.3 (2026-10-26)
https://github.com/cli/cli/releases/tag/v2.103.3
```

Рисунок 11 – Проверка версии GitHub CLI

Далее был создан Fork исходного репозитория BSUIR-SDLC/SDLC в личном аккаунте GitHub. Fork представляет собой независимую копию чужого репозитория, которая сохраняет связь с исходным проектом, но позволяет вносить собственные изменения без непосредственного воздействия на оригинал. В результате был создан публичный Fork radamonis-hub/SDLC, предназначенный для выполнения лабораторной работы и размещения разрабатываемого проекта (рисунок 12).

```
C:\Users\Ruslan>gh repo fork BSUIR-SDLC/SDLC --clone=false
✓ Created fork radamonis-hub/SDLC

C:\Users\Ruslan>gh repo view radamonis-hub/SDLC --json nameWithOwner,isFork,isPrivate,url,defaultBranchRef
{
  "defaultBranchRef": {
    "name": "main"
  },
  "isFork": true,
  "isPrivate": false,
  "nameWithOwner": "radamonis-hub/SDLC",
  "url": "https://github.com/radamonis-hub/SDLC"
}
```

Рисунок 12 – Создание Fork исходного репозитория

После создания Fork была подготовлена локальная рабочая директория, в которую выполнено клонирование собственного удалённого репозитория посредством SSH. Клонирование обеспечивает получение локальной копии проекта, включая его файлы, историю изменений и информацию об удалённых ветках. Использование SSH-адреса позволяет взаимодействовать с репозиторием через ранее настроенный механизм аутентификации (рисунок 13).

```
C:\Users\Ruslan>mkdir "%USERPROFILE%\Documents\Github"

C:\Users\Ruslan>cd /d "%USERPROFILE%\Documents\Github"

C:\Users\Ruslan\Documents\Github>git clone git@github.com:radamonis-hub/SDLC.git
Cloning into 'SDLC'...
Enter passphrase for key '/c/Users/Ruslan/.ssh/id_ed25519':
remote: Enumerating objects: 30, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 30 (delta 0), reused 0 (delta 0), pack-reused 22 (from 1)
Receiving objects: 100% (30/30), 74.57 KiB | 483.00 KiB/s, done.
```

Рисунок 13 – Создание локальной копии Fork репозитория

После завершения клонирования была проверена структура полученного репозитория. Для этого изучено содержимое рабочей директории и представление файлов в виде дерева каталогов. Были рассмотрены основные файлы и папки исходного проекта, включая документацию, шаблоны проектирования и материалы, связанные с требованиями и архитектурой программного обеспечения. Изучение структуры позволило определить расположение существующих материалов и подготовить репозиторий к дальнейшему внесению собственных изменений (рисунок 14).

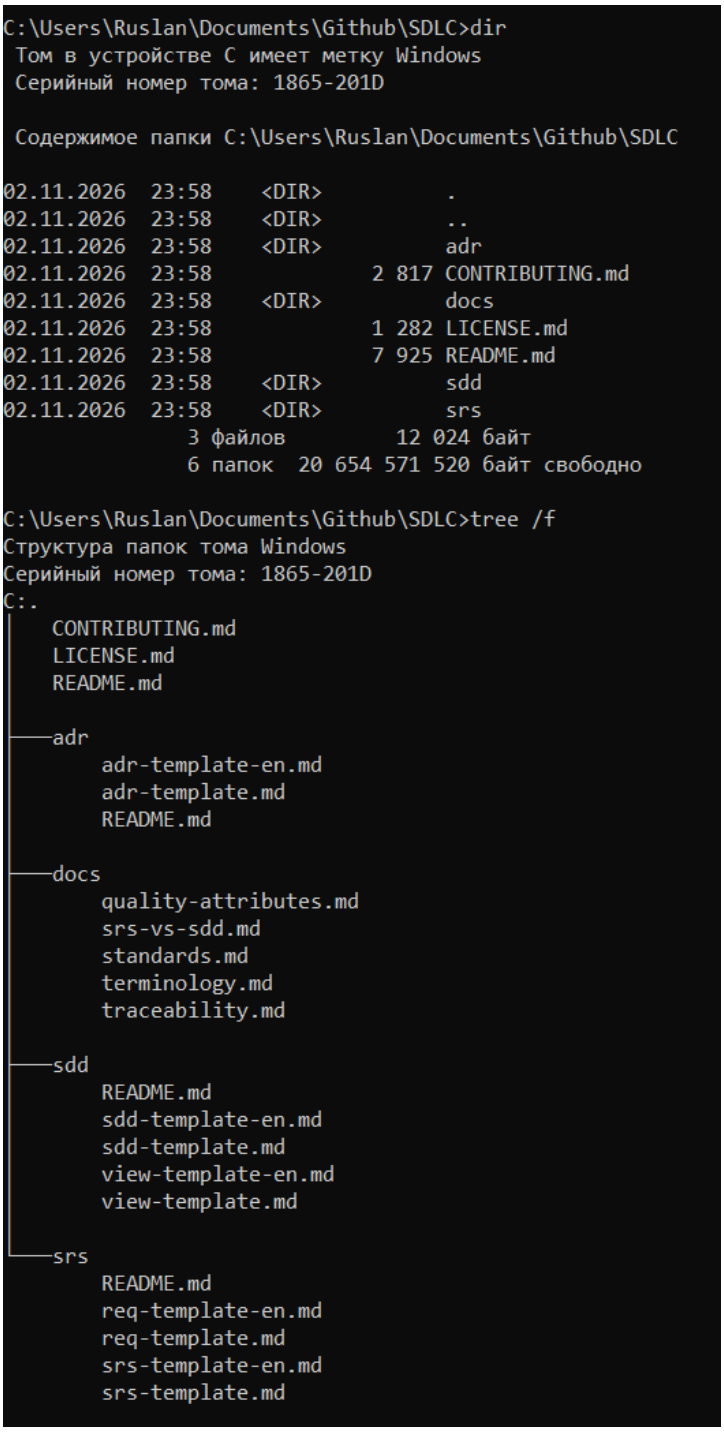


Рисунок 14 – Изучение структуры клонированного репозитория

Для выполнения первой лабораторной работы была создана отдельная локальная ветка lab1 на основе основной ветки main. Использование отдельных веток позволяет изолировать изменения, относящиеся к конкретному этапу разработки, сохраняя основную версию проекта неизменной. После создания ветки дальнейшие изменения выполнялись в её контексте (рисунок 15).

```
C:\Users\Ruslan\Documents\Github\SDLC>git switch -c lab1
Switched to a new branch 'lab1'

C:\Users\Ruslan\Documents\Github\SDLC>git branch
* lab1
  main
```

Рисунок 15 – Создание и переключение на ветку lab1

Следующим шагом стала настройка файла .gitignore, предназначенного для исключения из системы контроля версий временных и генерируемых файлов, не требующих хранения в репозитории. В него были внесены правила игнорирования служебных каталогов среды разработки, результатов сборки, скомпилированных файлов и системных данных. Это позволяет поддерживать репозиторий в упорядоченном состоянии и избегать добавления в него избыточных файлов. После создания файла было проверено содержимое рабочей директории (рисунок 16).

```
C:\Users\Ruslan\Documents\Github\SDLC>notepad .gitignore

C:\Users\Ruslan\Documents\Github\SDLC>dir /a
 Том в устройстве C имеет метку Windows
 Серийный номер тома: 1865-201D

 Содержимое папки C:\Users\Ruslan\Documents\Github\SDLC

03.11.2026  00:07    <DIR>          .
03.11.2026  00:07    <DIR>          ..
03.11.2026  00:06    <DIR>          .git
03.11.2026  00:08               154 .gitignore
02.11.2026  23:58    <DIR>          adr
02.11.2026  23:58             2 817 CONTRIBUTING.md
02.11.2026  23:58    <DIR>          docs
03.11.2026  00:05    <DIR>          lab1
02.11.2026  23:58             1 282 LICENSE.md
02.11.2026  23:58             7 925 README.md
02.11.2026  23:58    <DIR>          sdd
02.11.2026  23:58    <DIR>          srs
               4 файлов         12 178 байт
               8 папок  20 653 588 480 байт свободно
```

Рисунок 16 – Создание и настройка файла .gitignore

После настройки файла его изменения были добавлены в индекс Git – специальную область, в которой формируется набор изменений для последующего коммита. Проверка состояния репозитория показала, что файл .gitignore подготовлен к фиксации. Это позволило убедиться в корректности выбора изменений перед созданием коммита (рисунок 17).

```
C:\Users\Ruslan\Documents\Github\SDLC>git add .gitignore

C:\Users\Ruslan\Documents\Github\SDLC>git status
On branch lab1
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
```

Рисунок 17 – Добавление файла .gitignore в индекс Git

Подготовленные изменения были зафиксированы в истории репозитория с помощью коммита, после чего ветка lab1 была опубликована в удалённом Fork на GitHub. В процессе публикации локальная ветка была связана с соответствующей удалённой веткой. Дополнительная проверка состояния репозитория подтвердила успешную отправку изменений и отсутствие незакоммиченных файлов. Также была проверена информация о локальных и удалённых ветках, включая последние зафиксированные изменения (рисунок 18).

```
C:\Users\Ruslan\Documents\Github\SDLC>git push -u origin lab1
Enter passphrase for key '/c/Users/Ruslan/.ssh/id_ed25519':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 429 bytes | 214.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local objec
remote:
remote: Create a pull request for 'lab1' on GitHub by visiting:
remote:      https://github.com/radamonis-hub/SDLC/pull/new/lab1
remote:
To github.com:radamonis-hub/SDLC.git
 * [new branch]      lab1 -> lab1
branch 'lab1' set up to track 'origin/lab1'.

C:\Users\Ruslan\Documents\Github\SDLC>git status
On branch lab1
Your branch is up to date with 'origin/lab1'.

nothing to commit, working tree clean

C:\Users\Ruslan\Documents\Github\SDLC>git branch -vv
* lab1 cc85ef4 [origin/lab1] add gitignore configuration
  main 6c700a1 [origin/main] начальная версия репозитория
```

Рисунок 18 – Фиксация и публикация изменений ветки lab1

На заключительном этапе была подготовлена отдельная ветка ProjectTemplate, созданная непосредственно на основе основной ветки main. Она предназначена для размещения шаблона индивидуального проекта. В рамках данной ветки был подготовлен корневой файл README.md с указанием автора, названия дисциплины и описанием разрабатываемого приложения для определения гликемического индекса продуктов.

Кроме того, в репозитории был создан пустой проект на Java с использованием JavaFX и системы сборки Maven. Была настроена структура каталогов и подготовлен основной класс приложения, обеспечивающий запуск JavaFX-приложения. Дополнительно настроены файлы .gitignore, исключающие из отслеживания служебные и генерируемые данные среды разработки и процесса сборки.

После подготовки шаблона все необходимые изменения были зафиксированы в отдельном коммите. Затем ветка ProjectTemplate была отправлена в удалённый репозиторий GitHub и связана с соответствующей локальной веткой. Проверка состояния репозитория подтвердила успешную публикацию изменений и отсутствие незакоммиченных файлов (рисунок 19).

```
C:\Users\Ruslan\Documents\Github\SDLC>git commit -m "create JavaFX project template"
[ProjectTemplate c0d1c85] create JavaFX project template
 5 files changed, 133 insertions(+), 138 deletions(-)
 create mode 100644 .gitignore
 create mode 100644 GlycemicIndexApp/.gitignore
 create mode 100644 GlycemicIndexApp/pom.xml
 create mode 100644 GlycemicIndexApp/src/main/java/by/bsuir/glycemicindex/Main.java

C:\Users\Ruslan\Documents\Github\SDLC>git status
On branch ProjectTemplate
nothing to commit, working tree clean

C:\Users\Ruslan\Documents\Github\SDLC>git push -u origin ProjectTemplate
Enter passphrase for key '/c/Users/Ruslan/.ssh/id_ed25519':
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 16 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (14/14), 2.49 KiB | 212.00 KiB/s, done.
Total 14 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'ProjectTemplate' on GitHub by visiting:
remote:      https://github.com/radamonis-hub/SDLC/pull/new/ProjectTemplate
remote:
To github.com:radamonis-hub/SDLC.git
 * [new branch]      ProjectTemplate -> ProjectTemplate
branch 'ProjectTemplate' set up to track 'origin/ProjectTemplate'.
```

Рисунок 19 – Фиксация и публикация шаблона индивидуального проекта в ветке ProjectTemplate

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены принципы работы с системой контроля версий Git и освоены основные операции по управлению локальными и удалёнными репозиториями. Были выполнены установка и настройка необходимого программного обеспечения, настроена SSH-аутентификация, создан публичный Fork исходного репозитория и выполнено его клонирование.

В процессе работы получены практические навыки создания и переключения между ветками, настройки файла .gitignore, добавления изменений в индекс, фиксации изменений с помощью коммитов и их публикации на GitHub. Дополнительно был подготовлен шаблон индивидуального проекта на Java с использованием JavaFX и Maven.

В результате выполнения лабораторной работы закреплены навыки организации разработки с использованием Git, управления версиями программного проекта и взаимодействия с удалённым репозиторием GitHub.

КОНТРОЛЬНЫЕ ВОПРОСЫ

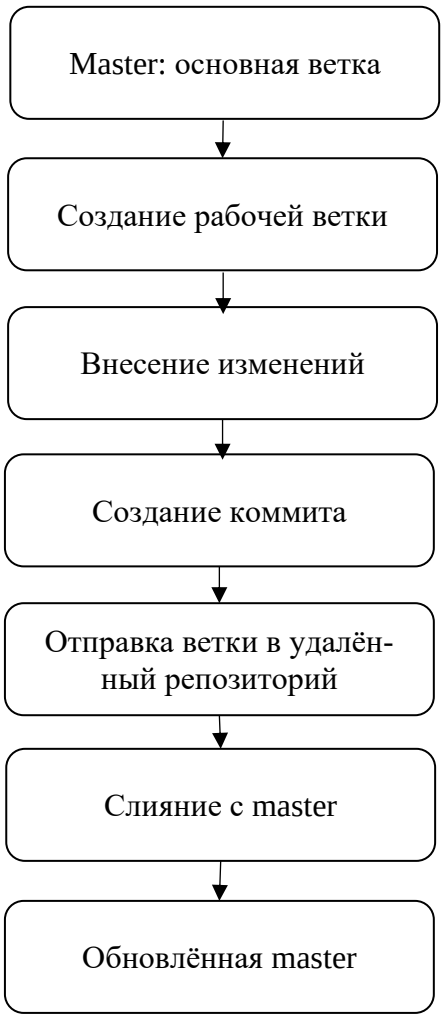
1. В чём преимущество Git перед SVN?

Git является распределённой системой контроля версий: каждый разработчик имеет полную локальную копию репозитория и его истории. Большинство операций выполняется локально, поэтому Git работает быстрее и позволяет работать без постоянного подключения к серверу. Также Git удобнее поддерживает параллельную разработку с помощью веток.

2. В чём преимущество распределённых репозиториев?

Каждый участник имеет полную копию репозитория, поэтому может работать независимо от центрального сервера. Это обеспечивает возможность автономной работы, ускоряет выполнение операций и повышает надёжность хранения истории изменений.

3. Нарисуйте граф типичной работы с веткой master в системе контроля версий Git.



4. Назовите наиболее часто используемые команды командной строки.

– git init – создание репозитория;
– git clone – клонирование репозитория;
– git status – просмотр состояния файлов;
– git add – добавление изменений в индекс;
– git commit – фиксация изменений;
– git push – отправка изменений в удалённый репозиторий;
– git pull – получение изменений из удалённого репозитория;
– git fetch – получение информации об удалённых изменениях;
– git branch – просмотр и создание веток;
– git switch – переключение между ветками;
– git merge – слияние веток;
– git log – просмотр истории коммитов.

5. Что нужно делать, если кто-то успел внести в удалённый репозиторий изменения?

Необходимо получить актуальные изменения из удалённого репозитория с помощью git pull, чтобы синхронизировать локальную копию с удалённой. Если возникнут конфликты слияния, их нужно разрешить вручную, проверить результат и зафиксировать изменения. После этого можно продолжать работу и отправлять собственные изменения.